

Politécnico de Leiria
Escola Superior de Tecnologia e Gestão
Departamento de Engenharia Eletrotécnica
Mestrado em Eng.^a Eletrotécnica

**RELATÓRIO PROJETO ELETRÓNICA
CONFIGURÁVEL - ASK, OOK**

PEDRO FERREIRA N^o2022106868

PAULO SOUSA N^o2022102341

Leiria, Julho de 2026

INTRODUÇÃO E OBJETIVOS GLOBAIS

Este relatório descreve o desenvolvimento da Proposta D, que implementa uma Modulação ASK/OOK Multiplexada na Frequência. O projeto está dividido em duas fases: o Projeto 1, que implementa o sistema em hardware estático usando Programmable Logic, e o Projeto 2, que integra o Processing System para dar controlo e configuração dinâmica ao sistema.

Os objetivos do sistema base incluem: implementar os moduladores ASK e OOK no fabric da FPGA (Pynq-Z2), aplicar a técnica de multiplexagem por divisão na frequência (FDM), controlar de forma independente o ganho de cada canal, e implementar uma secção que emula o meio físico de transmissão. Também é necessário um mecanismo de seleção dinâmica dos sinais intermédios, para simplificar a validação, encaminhando-os para a saída e para inspeção por Integrated Logic Analyzer (ILA).

A Figura 1.1 mostra o diagrama de blocos da arquitetura da Proposta D.

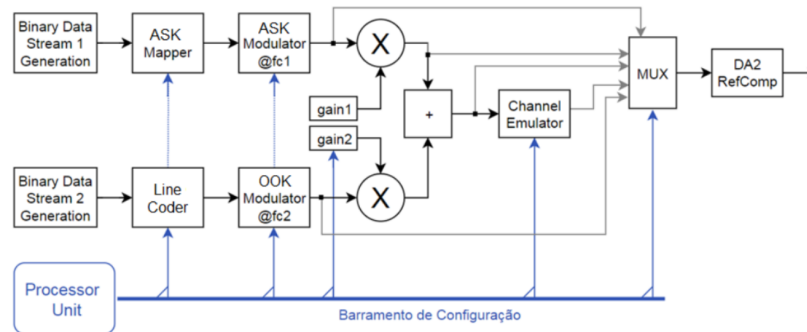


Figura 1.1: Diagrama da Proposta D.

PROJETO 1: IMPLEMENTAÇÃO EM HARDWARE PL

Nesta primeira fase, o foco foi implementar a arquitetura apenas em PL. O caminho do sinal, desde a geração até à saída, foi desenvolvido e testado no Vivado. A Figura 2.1 mostra a visão global da arquitetura implementada.

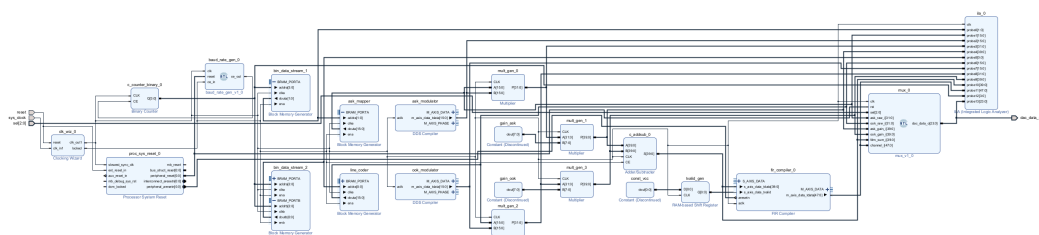


Figura 2.1: Arquitetura global do sistema.

2.1 GERAÇÃO DE DADOS E MODULADORES

A primeira parte do circuito gera os dados em banda base e mapeia-os em níveis de amplitude diferentes. Esta secção usa Block RAMs e compiladores DDS, como mostra a Figura 2.2.

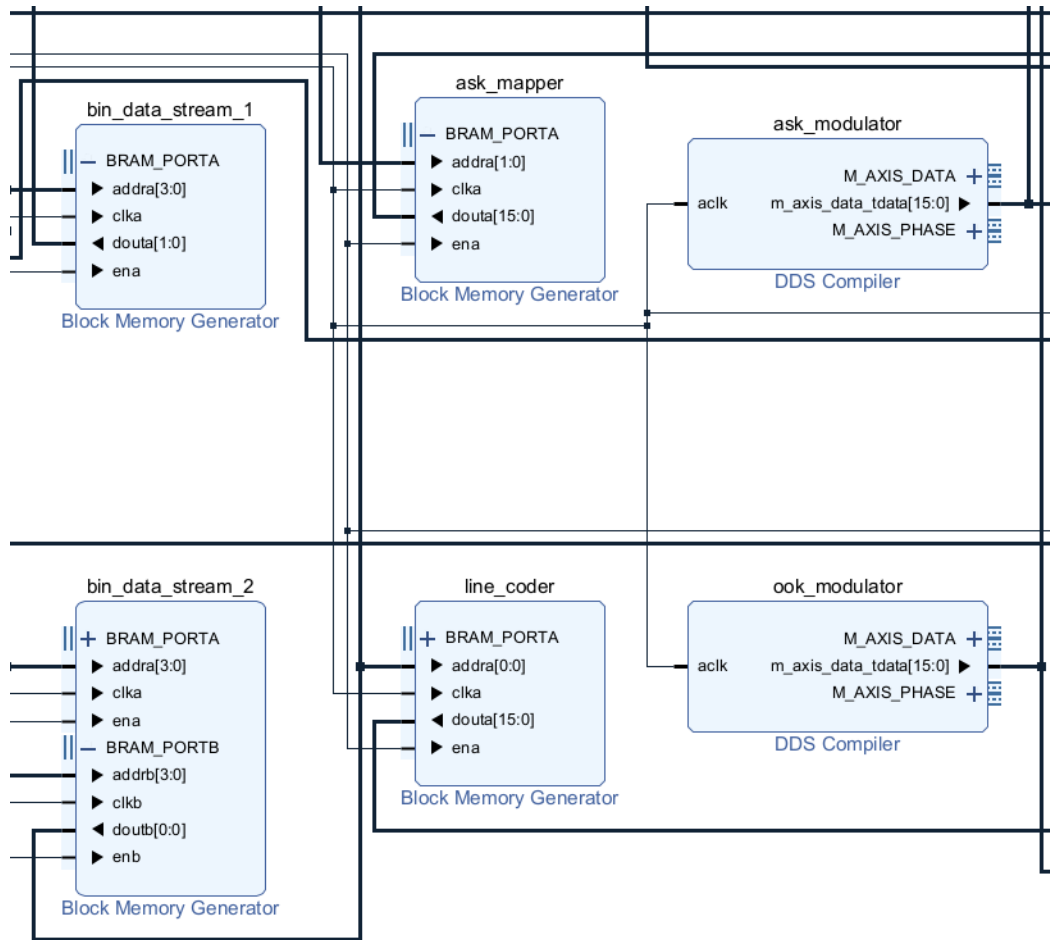


Figura 2.2: Secção de Geração de Dados com BRAMs e Compiladores DDS.

As Block RAMs funcionam como geradores pseudo aleatórios. O padrão cíclico de cada uma vem do ficheiro de inicialização usado para carregar a memória. Para o sinal ASK, a memória guarda valores em hexadecimal, com o vetor 0, 1, 2, 3, 0, 3, 1, 2, 1, 0, 3, 2, 3, 2, 1, 0. Para o OOK, a via é binária, com a sequência 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1.

Um bloco ASK Mapper converte esses valores nos 4 níveis de amplitude usados na modulação. O Line Coder converte o fluxo binário da via OOK. Neste Projeto 1, o nível do ASK vem só do ficheiro de inicialização e não se pode mudar em tempo real. O OOK está fixo ao código de linha NRZ.

A transmissão FDM precisa de portadoras sinusoidais para multiplicar o sinal em banda base. Estas são geradas por blocos Direct Digital Synthesizer (DDS): o ASK usa uma portadora a 1 MHz, e o OOK a 2 MHz.

2.2 GANHOS, MULTIPLICADORES E SOMADOR

Depois de geradas, as portadoras são multiplicadas pelos sinais das Block RAMs em blocos multiplicadores dedicados, o que completa a modulação em amplitude.

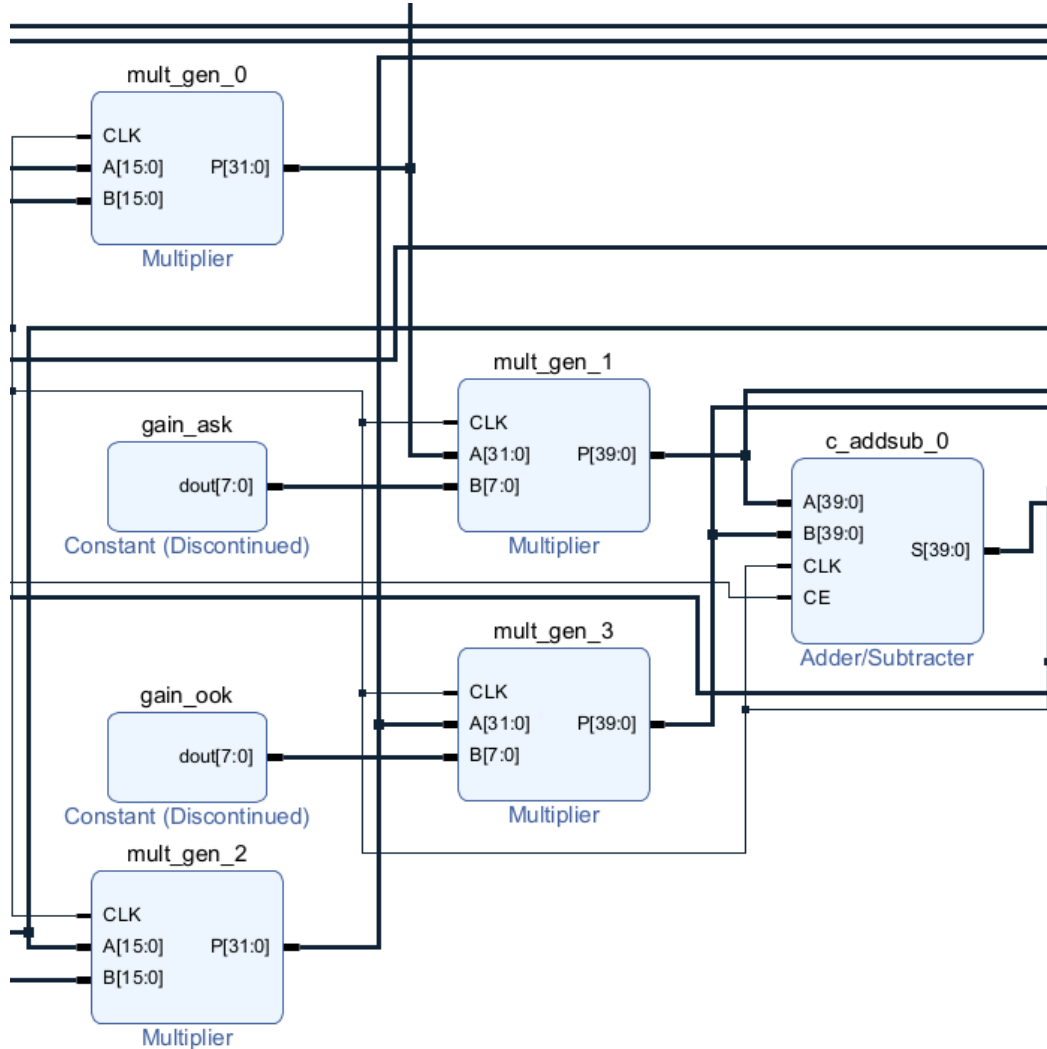


Figura 2.3: Multiplicadores, blocos de ganho e somador FDM.

O ganho de cada canal é controlado individualmente para equilibrar a potência na linha FDM, e este controle acontece na mesma etapa da modulação. Como nesta fase o sistema é todo em lógica estática, os ganhos vêm de blocos de constantes e não podem ser alterados dinamicamente.

A junção FDM é feita num somador digital, no domínio linear. Como o relógio principal corre a 125 MHz, foi necessário ativar o pipeline interno do somador, para aliviar a lógica e cumprir os requisitos de timing.

2.3 EMULAÇÃO DE CANAL E VISUALIZAÇÃO

Depois da transmissão, o sinal passa por um emulador de canal feito com um filtro FIR. Este IP corre em Single Rate, à taxa de amostragem de 125 MHz. O filtro serve para simular a atenuação de alta frequência que acontece num canal real, e foi desenhado com o algoritmo de Parks-McClellan. Tem 121 coeficientes, de 16 bits, com banda passante até 1.1 MHz e banda de rejeição a começar nos 1.9 MHz. Com isto, o canal ASK (1 MHz) passa sem distorção, enquanto o canal OOK (2 MHz) fica atenuado cerca de 32 dB, simulando as perdas do canal de transmissão.

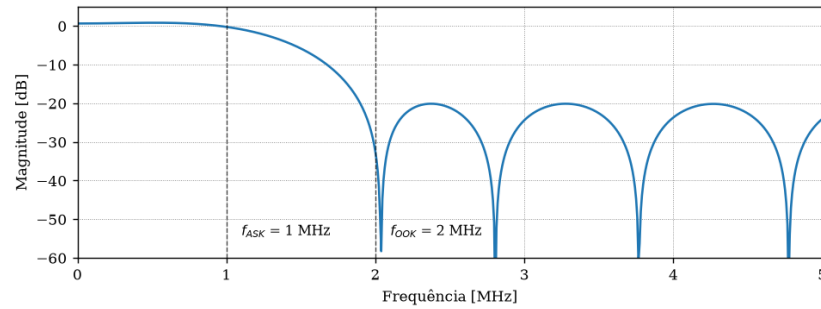


Figura 2.4: Resposta em frequência e impulso do filtro FIR calculado.

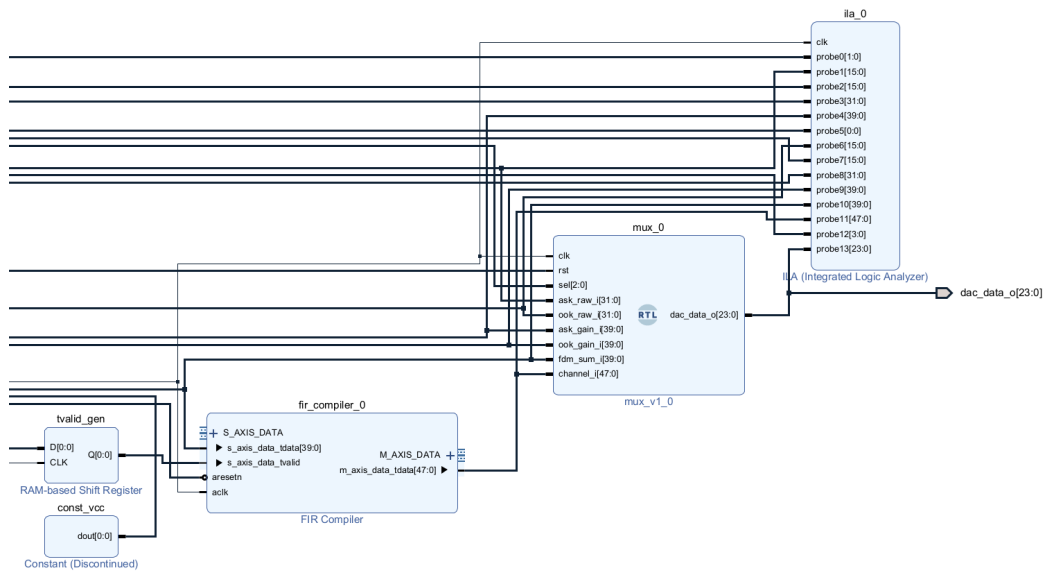


Figura 2.5: Filtro FIR de emulação de canal, MUX de visualização e ILA.

A multiplexagem de visualização foi materializada a partir de um MUX combinacional desenhado em VHDL para conduzir qualquer sinal requisitado ao barramento principal. O bloco DA2ref, que permitiria a averiguação prática via osciloscópio, estaria posicionado diretamente à frente do MUX caso estivesse implementado nesta fase. Atualmente o circuito apenas encaminha os dados e aciona a visualização em ferramentas de simulação.

Como exemplos demonstrativos, onde a linha laranja é a saída do MUX, as linhas vermelha e azul são o sinal ASK e OOK com ganhos, respetivamente, apresentam-se os resultados através da simulação:

Quando acionada a segunda entrada do MUX através do bit de seleção 010, observa-se a representação intermédia do sinal ASK na Figura 2.6.

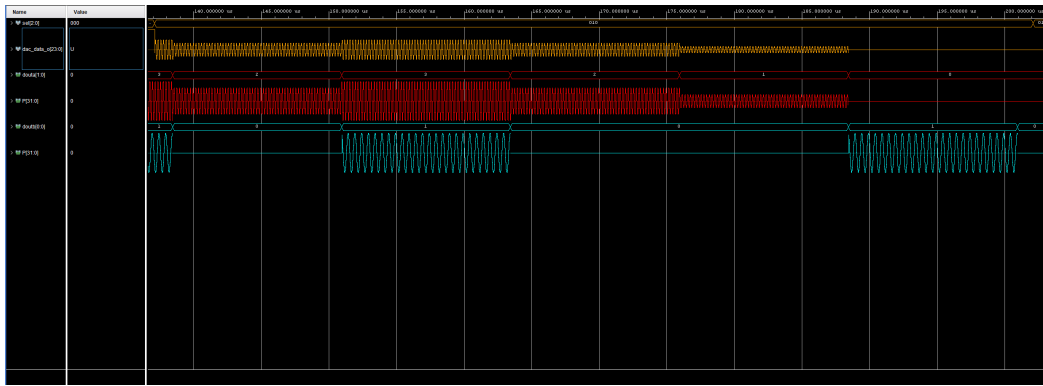


Figura 2.6: Visualização do estado lógico quando seleção é 010.

Ao transitar para a terceira entrada configurada com o bit 011, é possível verificar a via de sinal do modulador OOK, tal como se pode verificar na Figura 2.7.

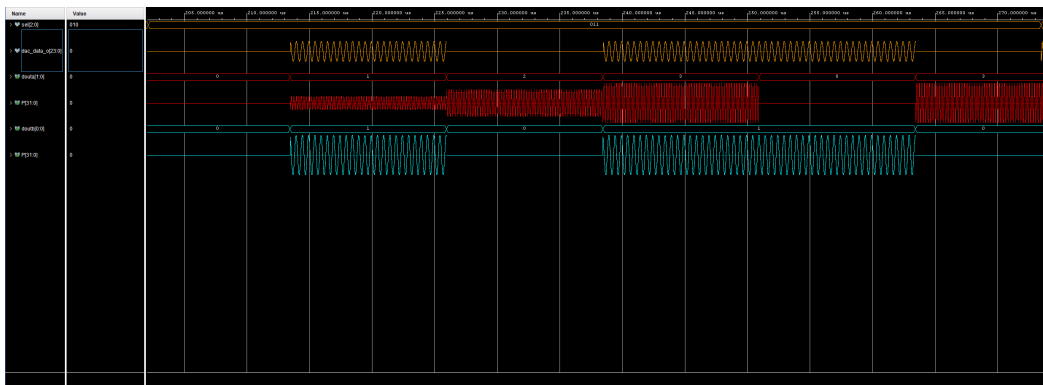


Figura 2.7: Visualização do estado lógico quando seleção é 011.

Ao seleccionar a quarta entrada (bit 100 na seleção), visualiza-se a via do sinal multiplexado conforme demonstra a Figura 2.8.

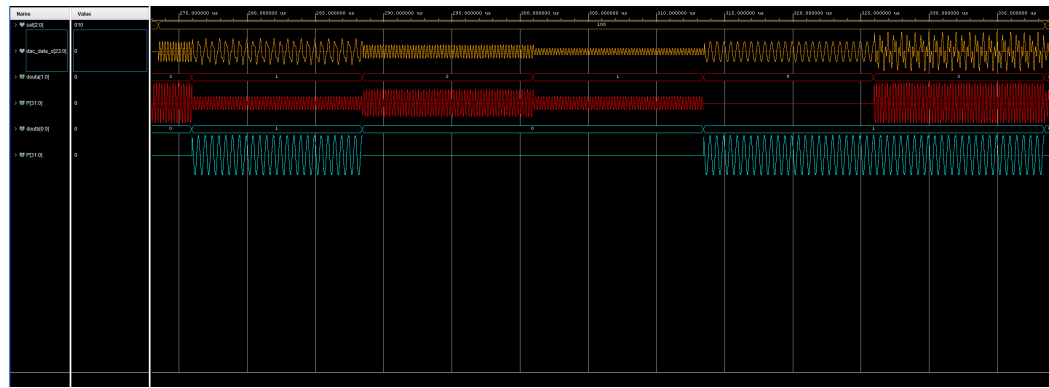


Figura 2.8: Visualização do estado lógico quando seleção é 100.

Selecionando a última entrada com o bit 101, vemos a componente final da via. Aqui dá para ver o efeito direto do filtro FIR do emulador de canal: na primeira trama, o sinal OOK é zero, então a saída do FIR permanece a mesma que ASK. No momento que existam dados na linha OOK, o FIR atenua fortemente o sinal OOK, permanecendo apenas o sinal ASK. Na trama final, observa-se a ligeira atenuação do sinal OOK quando ASK é zero. Esta diferença confirma o comportamento seletivo do canal, como se vê na Figura 2.9.

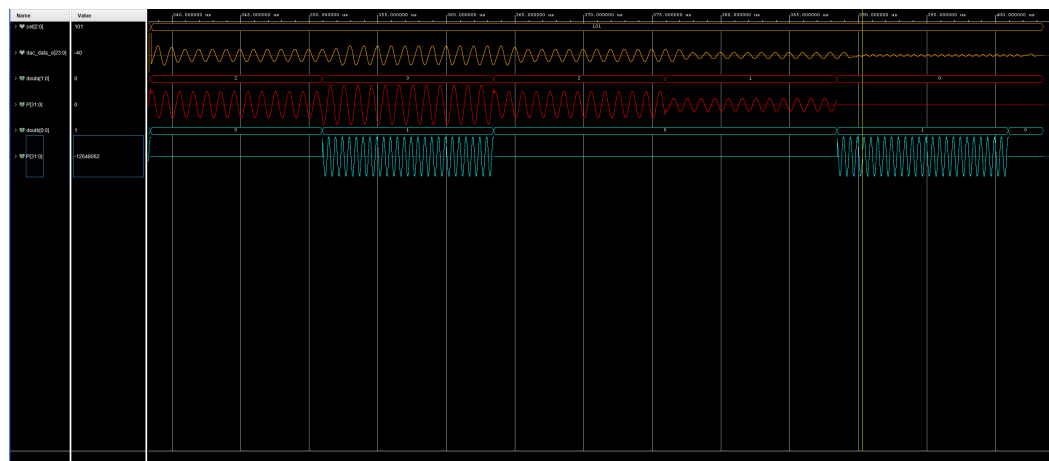


Figura 2.9: Visualização do estado lógico quando seleção é 101.

Todos os sinais chegam também a um Integrated Logic Analyzer (ILA), o que permite no futuro fazer leituras diretamente na placa. Os processos e mapeamentos foram verificados em testbench, confirmando que as modulações implementadas funcionam como previsto.

PROJETO 2: INTEGRAÇÃO DE HARDWARE E SOFTWARE PS

A segunda fase do projeto consiste na inserção do PS e na sua interligação com a PL. Este passo permite o controlo dos parâmetros de modulação e multiplexagem em tempo de execução.

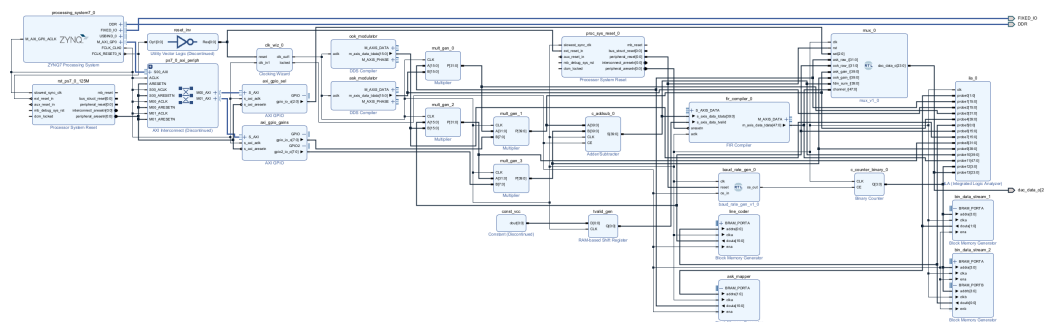


Figura 3.1: Diagrama de blocos do sistema.

3.1 SISTEMA DE PROCESSAMENTO E BARRAMENTO

O processador atua como master na comunicação. O IP `processing_system7_0` fornece o sinal de relógio principal `FCLK_CLK0`, de 125 MHz, e o sinal de reset de toda a parte lógica. O PS tem como base um processador com arquitetura ARM e interage com memórias e periféricos, corre o sistema operativo Linux. A comunicação entre o processador e a FPGA é feita através de um IP AXI Interconnect, que traduz comandos de memória do PS em transações AXI4-Lite dirigidas aos periféricos slave.

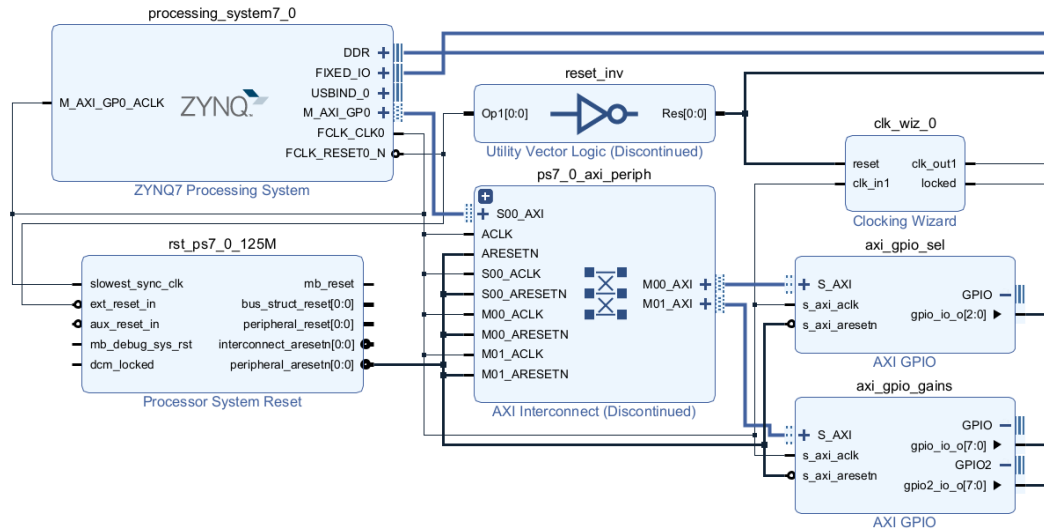


Figura 3.2: Detalhe do Zynq PS e interligação com os módulos AXI GPIO.

3.2 CONTROLADORES DE PERIFÉRICOS

Os blocos AXI GPIO funcionam como interface de modificação dos parâmetros dos caminhos de dados da modulação FDM, configurados como slaves AXI. O `axi_gpio_sel` tem um único canal, com largura de 3 bits, ligado à entrada de seleção `sel` do multiplexador. O `axi_gpio_gains` está configurado em dual channel, com os canais 1 e 2 a funcionar com largura de 8 bits: o canal 1 controla o ganho do ASK e o canal 2 o ganho do OOK.

O funcionamento interno dos GPIOs consiste em manter o valor recebido na porta AXI e na sua ligação aos pinos físicos ligados à lógica. A interação com o PS ocorre por Memory-Mapped I/O: cada escrita gera uma transação de 32 bits no barramento AXI, da qual apenas utiliza-mos o valor de 8 bits.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/processing_system7_0/Data)					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/axi_gpio_gains/S_AXI	S_AXI	Reg	0x4121_0000	64K	0x4121_FFFF
/axi_gpio_sel/S_AXI	S_AXI	Reg	0x4120_0000	64K	0x4120_FFFF

Figura 3.3: Mapeamento de memória no Address Editor do Vivado.

Os parâmetros configuráveis via GPIO são o ganho aplicado ao somador e o canal selecionado no multiplexador. Mudar estes valores muda o balanceamento de potência entre vias e o comportamento das saídas intermédias.

3.3 SINCRONIZAÇÃO E GERAÇÃO DE BAUD RATE

O relógio do sistema vem do PS e tem frequência de 125 MHz. As portadoras usadas andam na ordem de 1 MHz e 2 MHz. Se a sequência em banda base for lida a 125 MHz, esta sobrepõe-se à portadora e o sinal modulado sai distorcido. Por isso foi criado o bloco `baud_rate_gen.vhd`, que faz a divisão do relógio.

Este módulo gera uma nova baud rate para as memórias BRAM. As entradas são o relógio do sistema e um reset síncrono, a saída é um impulso digital. Por dentro, tem um contador síncrono que conta ciclos de relógio, quando chega aos 1250 ciclos, o impulso de saída fica a 1 lógico. Este bloco não fala com o barramento do PS. Com este divisor, a baud rate do sistema FDM fica em 100 kHz.

3.4 INTERFACE DE SOFTWARE PYNQ

O sistema é controlado a partir de uma aplicação Python na framework PYNQ. A aplicação carrega o bitstream `.bit` e o ficheiro com a descrição de hardware `.hwh`. A biblioteca `pynq.Overlay` serve para instanciar o overlay e aceder aos endereços dos módulos `axi_gpio_sel` e `axi_gpio_gains`.



Name	Last Modified	File size
..	seconds ago	
Untitled.ipynb	2 months ago	5.21 kB
fdm_wrapper.bit	an hour ago	4.05 MB
fdm_wrapper.hwh	2 hours ago	530 kB
loaded.xclbin	2 months ago	3.39 kB
test_fdm.py	2 months ago	889 B

Figura 3.4: Interface de gestão no ambiente Linux PYNQ.

```

In [1]: from pynq import Overlay
import time

In [2]: print("Loading Overlay...")
overlay = Overlay("fdm_wrapper.bit")
print("Overlay loaded successfully!")

Loading Overlay...
Overlay loaded successfully!

In [3]: print("Mapping AXI GPIO channels...")
axi_sel = overlay.axi_gpio_sel
axi_gains = overlay.axi_gpio_gains

sel_channel = axi_sel.channel1
ask_gain = axi_gains.channel1
ook_gain = axi_gains.channel2

Mapping AXI GPIO channels...

In [4]: print("Setting gains to maximum (255)...")
ask_gain.write(255, 0xFF)
ook_gain.write(255, 0xFF)

Setting gains to maximum (255)...
    
```

Figura 3.5: Escrita de registros AXI Lite via Jupyter Notebook.

Os scripts escrevem diretamente nas portas físicas através do barramento, preenchendo o resto do pacote com zeros de forma a agregar os 32 bits.

O controlo dinâmico do multiplexer e dos ganhos dos canais ASK e OOK foi testado no ambiente PYNQ, permitindo reconfigurar o sistema sem recompilar o hardware. No entanto, por instabilidades no ambiente de simulação e na interface do Vivado, não foi possível recolher e analisar as formas de onda em simultâneo com a variação destes parâmetros. Esta limitação foi agravada pela não implementação do bloco DAC na etapa final da cadeia de transmissão, o que impediu verificar os sinais com um osciloscópio físico.

CONCLUSÕES

O desenvolvimento da Proposta D exigiu um trabalho de integração significativo entre a lógica programável e o processador . Por limitações de tempo, não foi possível implementar todas as funcionalidades planeadas, como a interface DAC para emitir o sinal modulado e a validação final em placa. Ainda assim, o sistema desenvolvido até este ponto mostra que a arquitetura definida funciona como previsto.

O trabalho permitiu consolidar competências práticas de digital design entre PL e PS: desde a criação dos geradores e moduladores de sinal na FPGA até ao controlo de registos e parâmetros via barramento AXI no ambiente PYNQ . Isto cobre o objetivo principal da unidade curricular.

